Guía Técnica Exhaustiva de Replicación: Motor Geofísico e Inversión Subterránea 3D (`gpr3d`)

Manual de Tareas de Implementación en C++20, Justificación Física y Referencias Universitarias en Alemán

---

Proyecto: `SignalHandler` (*gpr3d*) • **Fecha:** Noviembre 2026 • **Estándar:** C++20 estricto

---

Contents

1 Visión General, Metodología y Arquitectura del Sistema **1**

1.1 Tabla Sinóptica de los 13 Sprints de Implementación 3

2 Sprint 0: Infraestructura C++20, CMake, Sanitizers y Entorno de Pruebas **3**

2.1 1. Fundamentos y Mapeo Bibliográfico 3

2.2 2. Justificación de Ingeniería: ¿Por qué hay que implementarlo? 3

2.3 3. Tareas Detalladas de Implementación 4

2.4 4. Prueba de Verificación Unitaria 5

3 Sprint 1: Núcleo Binario, Decodificación Endianness y Flotante IBM 360 **5**

3.1 1. Fundamentos y Mapeo Bibliográfico 5

3.2 2. Justificación de Ingeniería: ¿Por qué hay que implementarlo? 5

3.3 3. Tareas Detalladas de Implementación 6

3.4 4. Prueba de Verificación Unitaria 7

4 Sprint 2: Tipos de Dominio Geofísico y Simulador Ricker Sintético **7**

4.1 1. Fundamentos y Mapeo Bibliográfico 7

4.2 2. Justificación de Ingeniería: ¿Por qué hay que implementarlo? 7

4.3 3. Tareas Detalladas de Implementación 8

4.4 4. Prueba de Verificación Unitaria 9

5 Sprint 3: Geodesia Espacial (WGS84, ECEF, ENU, UTM y Lever-Arm) **9**

5.1 1. Fundamentos y Mapeo Bibliográfico 9

5.2 2. Justificación de Ingeniería: ¿Por qué hay que implementarlo? 9

5.3 3. Tareas Detalladas de Implementación 10

5.4 4. Prueba de Verificación Unitaria 10

6 Sprint 4: Receptores GNSS, NMEA-0183 (NMEA GGA) e Interpolación de Trayectoria **11**

6.1 1. Fundamentos y Mapeo Bibliográfico 11

6.2 2. Justificación de Ingeniería: ¿Por qué hay que implementarlo? 11

6.3 3. Tareas Detalladas de Implementación 11

6.4 4. Prueba de Verificación Unitaria 12

7 Sprint 5: Decodificadores I/O de Georradar (GSSI DZT, SEG-Y y MALA RD3) **12**

7.1 1. Fundamentos y Mapeo Bibliográfico 12

7.2 2. Justificación de Ingeniería: ¿Por qué hay que implementarlo? 13

7.3 3. Tareas Detalladas de Implementación 13

7.4 4. Prueba de Verificación Unitaria 14

8 Sprint 6: Núcleo Espectral FFT y Transformada Analítica de Hilbert **14**

8.1 1. Fundamentos y Mapeo Bibliográfico 14

8.2 2. Justificación de Ingeniería: ¿Por qué hay que implementarlo? 14

8.3 3. Tareas Detalladas de Implementación 15

8.4 4. Prueba de Verificación Unitaria 15

9 Sprint 7: Pipeline de Acondicionamiento DSP (Dewow, t0, BP, Fondo, SEC) **15**

9.1 1. Fundamentos y Mapeo Bibliográfico 15

9.2 2. Justificación de Ingeniería: ¿Por qué hay que implementarlo? 16

9.3 3. Tareas Detalladas de Implementación 17

9.4 4. Prueba de Verificación Unitaria 19

10 Sprint 8: Migración 2D de Kirchhoff y Nubes de Puntos 3D Georreferenciadas **19**

10.1 1. Fundamentos y Mapeo Bibliográfico 19

10.2 2. Justificación de Ingeniería: ¿Por qué hay que implementarlo? 20

10.3 3. Tareas Detalladas de Implementación 20

10.4 4. Prueba de Verificación Unitaria 20

11 Sprint 9: Inversión Geofísica Subterránea (Semblanza, Fresnel y ASTM D6432) **20**

11.1 1. Fundamentos y Mapeo Bibliográfico 20

11.2 2. Justificación de Ingeniería: ¿Por qué hay que implementarlo? 21

11.3 3. Tareas Detalladas de Implementación 21

11.4 4. Prueba de Verificación Unitaria 23

12 Sprint 10: Reconstrucción Semántica 3D y Generación de Mallas **23**

12.1 1. Fundamentos y Mapeo Bibliográfico 23
12.2 2. Justificación de Ingeniería: ¿Por qué hay que implementarlo? 24
12.3 3. Tareas Detalladas de Implementación 24
12.4 4. Prueba de Verificación Unitaria 25

13 Sprint 11: Exportadores e Interoperabilidad (glTF 2.0 Binary, GeoJSON 3D, HTML) 25

13.1 1. Fundamentos y Mapeo Bibliográfico 25
13.2 2. Justificación de Ingeniería: ¿Por qué hay que implementarlo? 25
13.3 3. Tareas Detalladas de Implementación 26
13.4 4. Prueba de Verificación Unitaria 26

14 Sprint 12: Integración CLI, Analítica Gráfica y Validación Real 26

14.1 1. Fundamentos y Mapeo Bibliográfico 26
14.2 2. Justificación de Ingeniería: ¿Por qué hay que implementarlo? 26
14.3 3. Tareas Detalladas de Implementación 27
14.4 4. Prueba de Verificación Unitaria 27

15 Conclusiones y Hoja de Ruta de Especialización Profesional 27

---

1 Visión General, Metodología y Arquitectura del Sistema

Esta guía técnica tiene como finalidad permitir la reconstrucción completa e independiente de la suite geofísica y de procesamiento de señales `gpr3d` (`SignalHandler`). A diferencia de resúmenes teóricos superficiales, este documento detalla:

1. **Por qué hay que implementarlo:** Justificación física electromagnética y computacional de cada módulo, explicando qué fallos ocurren en los datos o en memoria si se omite o diseña incorrectamente.
2. **Mapeo académico en alemán:** Localización precisa de temas en los libros estándar *Mathematik* (Tilo Arens et al.), *Physik für Wissenschaftler und Ingenieure* (Tipler & Mosca) y *C++ professionell anwenden* (Ulrich Breymann).
3. **Tareas de implementación paso a paso:** Estructuras de datos, atributos, firmas de métodos, ecuaciones en pseudocódigo y código C++20 real.
4. **Criterios de verificación unitaria:** Casos de prueba reproducibles con entradas numéricas y tolerancias de error ϵ.

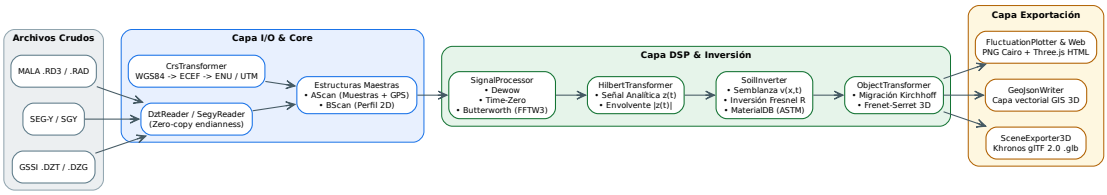


Figure 1: Arquitectura en capas y flujo de transformaciones del motor geofísico `gpr3d`.

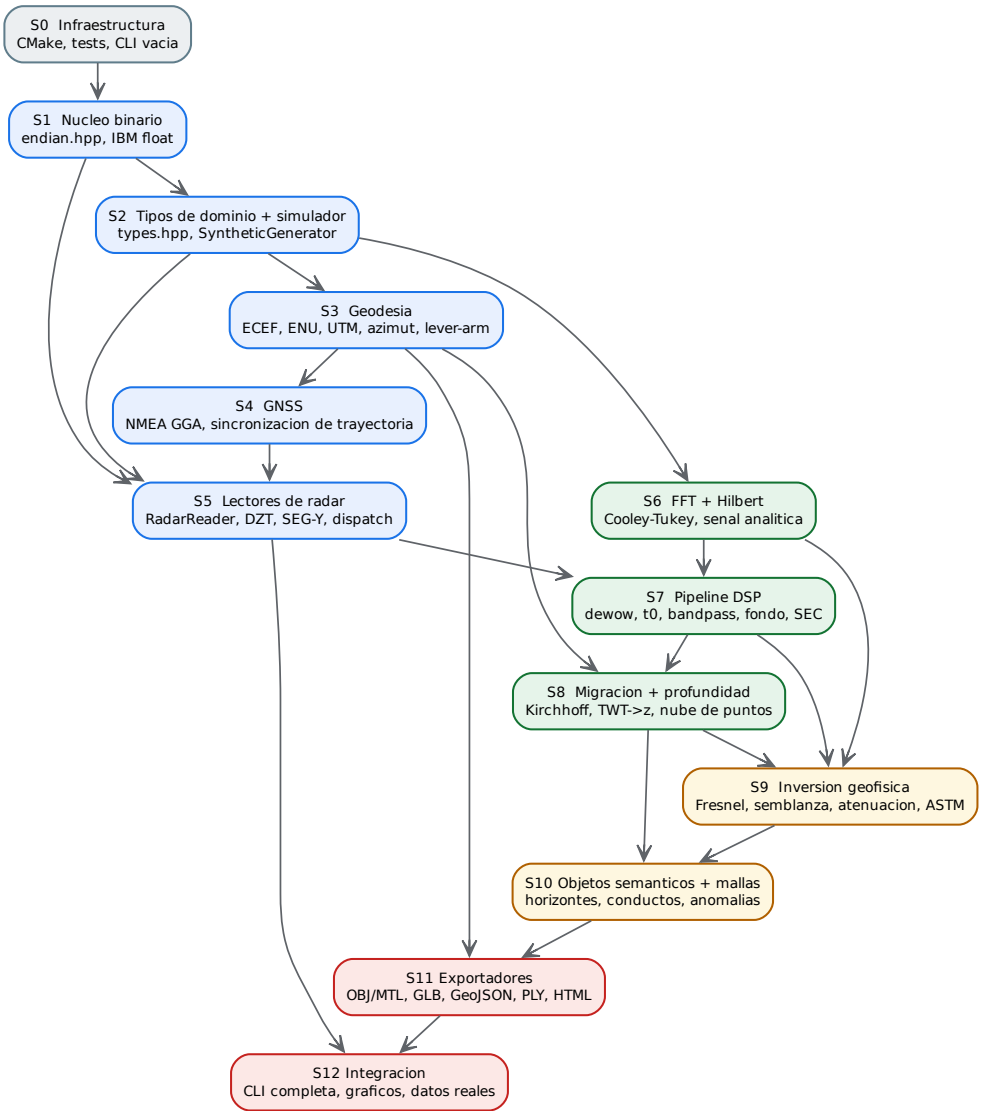


Figure 2: Grafo de dependencias técnicas de los 13 sprints de implementación progresiva.

Sprint	Módulo y Objeto Técnico	*Mathematik* (Arens)	*Physik* (Tipler)	*C++20* (Breymann)
0	Infraestructura CMake y Sanitizers	–	–	Kap. 1 & 21 (Toolchain, ASan)
1	Núcleo binario, Endianness, IBM 360	Kap. 1 & 28 (Numerik)	Kap. 1 (Messung)	Kap. 2, 3 (`<bit>`, Bitwise)
2	Tipos de Dominio y Simulador Ricker	Kap. 1 & 28 (Matrizen)	Kap. 15 (Wellen & TWT)	Kap. 7 & 8 (RAII, Move)
3	Geodesia WGS84, ECEF, ENU, UTM	Kap. 7 & 8 (Vektoren, Matrizen)	Kap. 3 (Kinematik 3D)	Kap. 7 (`std::optional`)
4	GNSS, NMEA GGA y Sincronización	Kap. 28 (Interpolation)	Kap. 3 (Trajektorie)	Kap. 16 (Stream-Parsing)
5	Lectores DZT, SEG-Y y MALA RD3	Kap. 1 & 28	Kap. 15 (TWT)	Kap. 16 (Binäre E/A)
6	FFT Cooley-Tukey y Hilbert	Kap. 18 (Fourier, Hilbert)	Kap. 15 & 30 (Energie)	Kap. 14 (`std::complex`)
7	Pipeline DSP (Dewow, t0, BP, Fondo, SEC)	Kap. 18 (Faltung)	Kap. 15 & 28 (Filter)	Kap. 11 (`std::span`)
8	Migración Kirchhoff 2D y Nubes 3D	Kap. 24 & 28	Kap. 15 (Huygens)	Kap. 7 & 8 (Point Cloud)
9	Inversión: Semblanza, Fresnel, ASTM	Kap. 28 (Ausgleichsrechnung)	Kap. 30–32 (Optik, EM)	Kap. 13 & 14 (`nth_element`)
10	Mallas 3D y Triedro Frenet-Serret	Kap. 24 & 25 (Kurven 3D)	Kap. 15 (Wellenbeugung)	Kap. 7 & 8 (Mallas indexadas)
11	Exportadores GLB 2.0, GeoJSON, HTML	Kap. 7 (Projektionen)	Kap. 31 (PBR-Optik)	Kap. 16 (Serialización binaria)
12	CLI, Gráficos Cairo y Datos Reales	Kap. 28	Kap. 15	Kap. 1 (CLI & Integración)

Table 1: Mapeo curricular exhaustivo de los 13 sprints de implementación.

1.1 Tabla Sinóptica de los 13 Sprints de Implementación

2 Sprint 0: Infraestructura C++20, CMake, Sanitizers y Entorno de Pruebas

2.1 1. Fundamentos y Mapeo Bibliográfico

- ***C++ professionell anwenden*** **(Ulrich Breymann)**:
  - **Kapitel 1 (*Entwicklungsumgebung und Compiler-Optionen*)**: Configuración del estándar C++20 (`-std=c++20`), flags de advertencia estrictas (`-Wall -Wextra -Wpedantic -Wconversion`) y estructura modular con CMake.
  - **Kapitel 21 (*Fehlersuche und Code-Qualität*)**: Detección temprana de errores de punteros, desbordamientos de búfer y conversiones con signo/sin signo mediante sanitizers de LLVM/GCC: AddressSanitizer (`-fsanitize=address`) y UndefinedBehaviorSanitizer (`-fsanitize=undefined`).

2.2 2. Justificación de Ingeniería: ¿Por qué hay que implementarlo?

El procesamiento de georradar involucra la lectura de millones de muestras binarias crudas desde disco hacia búferes continuos de memoria, y la ejecución de convoluciones y transformadas FFT. Un solo error de desalineación de punteros, un acceso fuera de límites (*out-of-bounds index*) o una conversión implícita de entero signado a no signado provocará:

1. Corrupción silenciosa de datos geofísicos (valores fantasma en el espectro o en las coordenadas).
2. Fallos de segmentación (*segmentation faults*) en tiempo de ejecución difíciles de reproducir al procesar perfiles gigantescos de más de 10 000 trazas.
3. Comportamiento indefinido (*Undefined Behavior*) dependiente de la optimización del compilador (`-O2` vs `-O3`).

Configurar un entorno estricto con sanitizers en el Sprint 0 garantiza que cualquier fallo de punteros se detecte de inmediato con un mensaje exacto de línea y llamada en la traza de pila (*stack trace*).

2.3 3. Tareas Detalladas de Implementación

- **Subtarea 0.1: Configuración de `CMakeLists.txt`:** Configura C++20 requerido, opciones condicionales de aceleración (`GPR3D_USE_FFTW3`, `GPR3D_USE_CAIRO`, `GPR3D_USE_OPENGL`) y flags de compilación para modo Debug y Release:

```
cmake_minimum_required(VERSION 3.20)
project(gpr3d VERSION 2.0.0 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

# Opciones de bibliotecas opcionales
option(GPR3D_USE_FFTW3 "Aceleracion espectral con FFTW3" ON)
option(GPR3D_USE_CAIRO "Renderizado grafico con Cairo" ON)
option(GPR3D_BUILD_TESTS "Compilar suite de pruebas unitarias" ON)

# Flags de advertencia estrictas y sanitizers en Debug
if (CMAKE_CXX_COMPILER_ID MATCHES "GNU|Clang")
    add_compile_options(-Wall -Wextra -Wpedantic -Wconversion)
    set(CMAKE_CXX_FLAGS_DEBUG "${CMAKE_CXX_FLAGS_DEBUG} -fsanitize=address,undefined -g")
    set(CMAKE_EXE_LINKER_FLAGS_DEBUG "${CMAKE_EXE_LINKER_FLAGS_DEBUG} -fsanitize=address,u
endif()
```

- **Subtarea 0.2: Estructura física del repositorio:** Crea las carpetas estándar: `include/gpr3d/core/`, `include/gpr3d/io/`, `include/gpr3d/dsp/`, `include/gpr3d/geo/`, `include/gpr3d/scene/`, `src/`, `apps/` y `tests/`.
- **Subtarea 0.3: Harness básico de pruebas:** Crea un ejecutable `tests/test_gpr3d.cpp` enlazado a la librería con una macro sencilla de aserción numérica:

```
#include <iostream>
#include <cmath>
#include <cstdlib>

#define ASSERT_NEAR(a, b, eps) \
    if (std::abs((a) - (b)) > (eps)) { \
        std::cerr << "Fallo en linea " << __LINE__ << ": " << (a) << " != " << (b) << std:
        std::exit(EXIT_FAILURE); \
    }
```

2.4 4. Prueba de Verificación Unitaria

Ejecuta `cmake -B build -DCMAKE_BUILD_TYPE=Debug && cmake --build build`. Verifica que compila con cero advertencias y que el ejecutable de pruebas termina con código de salida 0.

3 Sprint 1: Núcleo Binario, Decodificación Endianness y Flotante IBM 360

3.1 1. Fundamentos y Mapeo Bibliográfico

- ***Mathematik* (Tilo Arens et al.)**:
  - **Kapitel 1 (*Mengen und Zahlen*)**: Sistemas posicionales binario y hexadecimal, complemento a dos, máscaras de bits y aritmética de precisión finita.
  - **Kapitel 28 (*Numerik*)**: Estructura de números en punto flotante (signo, exponente sesgado y mantisa fraccionaria), sesgo (*bias*) y resolución de máquina.
- ***Physik* (Tipler & Mosca)**:
  - **Kapitel 1 (*Physikalische Größen und Messung*)**: Escalas de tiempo en nanosegundos (1 ns = 10^{-9} s) y frecuencias en megahertz (1 MHz = 10^6 Hz).
- ***C++ professionell anwenden* (Breymann)**:
  - **Kapitel 2 & 3 (*Datentypen, Bit-Operationen*)**: Tipos de ancho fijo (`uint8_t`, `uint16_t`, `uint32_t`, `int32_t`), operadores bitwise (`<<`, `>>`, `&`, `|`), y la biblioteca `<bit>` (`std::endian::native`).

3.2 2. Justificación de Ingeniería: ¿Por qué hay que implementarlo?

1. **Heterogeneidad de Endianness en Fabricantes de Georradar:** Los archivos GSSI RADAN (`.dzt`) guardan enteros en *Little-Endian* (byte menos significativo primero), mientras que los archivos SEG-Y clásicos provenientes de cintas sísmicas o georradar industrial están en *Big-Endian* (byte más significativo primero). Si un desarrollador intenta leer un entero de 16 bits directamente con `*(uint16_t*)ptr`, el código fallará en procesadores con convención contraria o causará una violación de alineación (*unaligned access*) en CPUs ARM.
2. **Incompatibilidad de Punto Flotante IBM 360 frente a IEEE 754:** El estándar SEG-Y histórico almacena amplitudes en flotantes de IBM System/360. El formato IBM utiliza **base 16** (hexadecimal) con un sesgo de 64 en el exponente:

$$x = (-1)^s \cdot 16^{c-64} \cdot \left(\sum_{i=1}^{24} b_{24-i} 2^{-i} \right) \tag{1}$$

   mientras que el estándar IEEE 754 moderno utiliza base 2 con sesgo 127 y un bit 1 implícito en la mantisa. Interpretar directamente los bits de un flotante IBM como flotante IEEE produce amplitudes erróneas de magnitudes absurdas ($\approx 10^{35}$) o valores `NaN`, destruyendo por completo cualquier cálculo posterior de energía o reflectancia.

3.3 3. Tareas Detalladas de Implementación

Crea el archivo `include/gpr3d/core/endian.hpp`:

- **Subtarea 1.1: Lectura segura de enteros Little-Endian y Big-Endian:**

```
namespace gpr3d::detail {
    inline uint16_t read_u16_le(const uint8_t* p) noexcept {
        return static_cast<uint16_t>(p[0]) | (static_cast<uint16_t>(p[1]) << 8);
    }
    inline uint32_t read_u32_le(const uint8_t* p) noexcept {
        return static_cast<uint32_t>(p[0]) |
               (static_cast<uint32_t>(p[1]) << 8) |
               (static_cast<uint32_t>(p[2]) << 16) |
               (static_cast<uint32_t>(p[3]) << 24);
    }
    inline int32_t read_i32_le(const uint8_t* p) noexcept {
        return static_cast<int32_t>(read_u32_le(p));
    }
    inline float read_f32_le(const uint8_t* p) noexcept {
        uint32_t u = read_u32_le(p);
        float f;
        std::memcpy(&f, &u, sizeof(float));
        return f;
    }
    inline uint16_t read_u16_be(const uint8_t* p) noexcept {
        return static_cast<uint16_t>((p[0] << 8) | p[1]);
    }
    inline uint32_t read_u32_be(const uint8_t* p) noexcept {
        return (static_cast<uint32_t>(p[0]) << 24) |
               (static_cast<uint32_t>(p[1]) << 16) |
               (static_cast<uint32_t>(p[2]) << 8) |
               static_cast<uint32_t>(p[3]);
    }
    inline int32_t read_i32_be(const uint8_t* p) noexcept {
        return static_cast<int32_t>(read_u32_be(p));
    }
    inline float read_f32_be(const uint8_t* p) noexcept {
        uint32_t u = read_u32_be(p);
        float f;
        std::memcpy(&f, &u, sizeof(float));
        return f;
    }
}
```

- **Subtarea 1.2: Decodificador IBM 360 Float a IEEE 754:**

```
inline double ibm_to_double(uint32_t ibm_bits) noexcept {
    if ((ibm_bits & 0x7FFFFFFF) == 0) return 0.0;
    const int sign = (ibm_bits & 0x80000000) ? -1 : 1;
    const int exponent = static_cast<int>((ibm_bits >> 24) & 0x7F) - 64;
    const uint32_t fraction = ibm_bits & 0x00FFFFFF;
    const double frac_val = static_cast<double>(fraction) / 16777216.0; // 2^24

    double factor = 1.0;
    if (exponent >= 0) {
        for (int i = 0; i < exponent; ++i) factor *= 16.0;
```

```
        } else {
            for (int i = 0; i < -exponent; ++i) factor /= 16.0;
        }
        return sign * frac_val * factor;
    }
```

- **Subtarea 1.3: Escritura de enteros en Little/Big Endian:** Implementa `write_u16_le`, `write_u32_le`, `write_u16_be`, `write_u32_be` para permitir la exportación de archivos glTF y DZT sintetizados.

3.4 4. Prueba de Verificación Unitaria

En `tests/test_endian.cpp`:

1. `read_u16_le({0x34, 0x12})` debe ser exactamente 4660 (`0x1234`).
2. `read_u16_be({0x12, 0x34})` debe ser exactamente 4660 (`0x1234`).
3. `ibm_to_double(0x41100000)` debe retornar exactamente 1.0 (signo +, exp $65 - 64 = 1$, fracc $1/16$, valor $16^1 \times 1/16 = 1.0$).
4. `ibm_to_double(0xC2760000)` debe retornar exactamente -118.0.

4 Sprint 2: Tipos de Dominio Geofísico y Simulador Ricker Sintético

4.1 1. Fundamentos y Mapeo Bibliográfico

- ***Mathematik* (Arens et al.)**:
  - **Kapitel 18 (*Fourier-Transformation*)**: Función gaussiana y sus derivadas temporales.
  - **Kapitel 28 (*Numerik*)**: Muestreo discreto temporal $t_j = j \cdot \Delta t$, discretización espacial de perfiles $x_i = i \cdot \Delta x$.
- ***Physik* (Tipler & Mosca)**:
  - **Kapitel 15 (*Wellen*)**: Propagación de pulsos, tiempo de viaje en dos sentidos (TWT) $t = 2z/v$, principio de superposición lineal y velocidad electromagnética $v = c_0/\sqrt{\varepsilon_r}$.
- ***C++ professionell anwenden* (Breymann)**:
  - **Kapitel 7 & 8 (*Klassen, Move-Semantik*)**: Objetos semánticos RAII, prevención de copias innecesarias con constructores por movimiento (`noexcept`), reserva previa de capacidad con `std::vector::reserve`.

4.2 2. Justificación de Ingeniería: ¿Por qué hay que implementarlo?

1. **Abstracción Agnóstica del Formato:** La lógica de procesamiento de señales (Dewow, Kirchhoff, Semblanza) no debe saber si los datos provinieron de un radar GSSI, MALA o SEG-Y. Se requiere una representación canónica en memoria (`BScan`) que contenga trazas verticales (`AScan`) y metadatos unificados.

2. **Indispensabilidad de un Generador Sintético para Verificación:** En geofísica de campo, los datos reales contienen ruido aleatorio, acoplamientos y geometrías desconocidas a priori. Para demostrar rigurosamente que los algoritmos de migración e inversión dieléctrica funcionan, es obligatorio disponer de un generador analítico sintético que cree radargramas controlados con ondículas de Ricker:

$$\psi(t) = \left(1 - 2\pi^2 f_0^2 (t - t_0)^2\right) \exp\left(-\pi^2 f_0^2 (t - t_0)^2\right) \tag{2}$$

donde se conocen exactamente las coordenadas del reflector (x_0, z_0) y la permitividad del estrato ε_r.

4.3 3. Tareas Detalladas de Implementación

- **Subtarea 2.1: Definición de `include/gpr3d/core/types.hpp`:** Declara las constantes físicas fundamentales y los contenedores de datos:

```
namespace gpr3d {
    inline constexpr double kSpeedOfLightMPerNs = 0.299792458; // c0 en m/ns

    struct AScan {
        std::size_t trace_id{0};
        double distance_along_track_m{0.0};
        std::optional<double> timestamp_sec;
        std::optional<Wgs84Coord> position_wgs84;
        std::optional<EnuCoord> position_enu;
        double heading_deg{0.0};
        std::vector<double> samples;
        std::vector<double> envelope;
    };

    struct SurveyMetadata {
        std::string source_format{"UNKNOWN"};
        double sample_interval_ns{0.1};
        double time_window_ns{50.0};
        double time_zero_ns{0.0};
        std::size_t samples_per_trace{0};
        double trace_spacing_m{0.02};
        double scans_per_second{50.0};
        double antenna_freq_mhz{400.0};
        double relative_permittivity{9.0};
    };

    struct BScan {
        SurveyMetadata metadata{};
        std::vector<AScan> traces{};
        [[nodiscard]] std::size_t num_traces() const noexcept { return traces.size(); }
        [[nodiscard]] std::size_t num_samples() const noexcept {
            return traces.empty() ? metadata.samples_per_trace : traces.front().samples.si
        }
        [[nodiscard]] bool empty() const noexcept { return traces.empty(); }
    };
}
```

- **Subtarea 2.2: Implementación de `SyntheticGenerator`:** En `src/sim/synthetic_generator.cpp`, implementa la generación de hipérbolas difractivas: Para cada traza en x, calcula la distancia

euclidiana al blanco $R = \sqrt{(x - x_0)^2 + z_0^2}$, el tiempo doble $t_{\text{twt}} = 2R/v$, y evalúa la ondícula de Ricker sobre la malla de muestras temporales.

4.4 4. Prueba de Verificación Unitaria

Genera una encuesta sintética con 100 trazas, $\Delta x = 0.05$ m, $\Delta t = 0.1$ ns, $\varepsilon_r = 9.0$ ($v = 0.10$ m/ns) y un conducto ubicado en $x = 2.5$ m, $z = 0.8$ m. Comprueba que en la traza central ($x = 2.5$ m), el pico máximo de amplitud se ubica exactamente en la muestra correspondiente a $t_0 = 2 \times 0.8/0.10 = 16.0$ ns (muestra $j = 160$).

5 Sprint 3: Geodesia Espacial (WGS84, ECEF, ENU, UTM y Lever-Arm)

5.1 1. Fundamentos y Mapeo Bibliográfico

- ***Mathematik* (Arens et al.)**:
  - **Kapitel 7 & 8 (*Vektoren und Matrizen*)**: Coordenadas cartesianas tridimensionales, producto cruz, matrices de rotación ortogonales ($R^{-1} = R^T$, $\det R = 1$) y cambio de sistema de coordenadas.
- ***Physik* (Tipler & Mosca)**:
  - **Kapitel 3 (*Bewegung in 3D*)**: Cinemática tridimensional, traslación de marcos de referencia y vectores de posición en marcos inerciales vs. móviles.
- ***C++ professionell anwenden* (Breymann)**:
  - **Kapitel 7 (*Klassen*)**: Estructuras de datos geométricas simples, métodos `noexcept` y constantes de `<numbers>` (`std::numbers::pi`).

5.2 2. Justificación de Ingeniería: ¿Por qué hay que implementarlo?

1. **No-Euclidianidad de Coordenadas Lat/Lon:** Las coordenadas GNSS angulares en grados (φ, λ) son esféricas/elipsoidales y distorsionan las distancias métricas según la latitud geográfica. A 50° N, 1° de longitud equivale a ≈ 71 km, mientras que 1° de latitud equivale a ≈ 111 km. Intentar calcular mallas 3D tubulares o distancias entre trazas directamente en grados produce cilindros aplastados y cálculos de velocidad completamente erróneos.
2. **Necesidad de Coordenadas Métricas Locales ENU:** Los motores 3D (WebGL, Three.js, glTF) y los algoritmos de migración necesitan coordenadas métricas ortonormales euclidianas. El sistema plano tangente local ENU (East-North-Up) proporciona un origen métrico $(0, 0, 0)$ donde el eje X es Este, el eje Y es Norte y el eje Z es la elevación ortométrica vertical.
3. **Corrección de Brazo de Palanca (*Lever-Arm*):** En carritos de georradar, el receptor GPS se instala en un mástil elevado, desplazado longitudinalmente (Δx_{fwd}) y transversalmente (Δy_{right}) respecto a la antena de radar sobre el suelo. Si no se compensa este desplazamiento con el rumbo instantáneo (ψ), los perfiles adquiridos en pasadas de ida y vuelta quedarán desalineados hasta 2 metros entre sí.

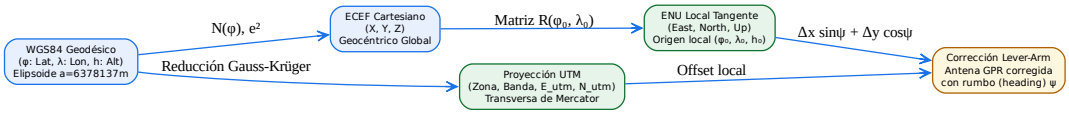


Figure 3: Cadena de transformaciones geodésicas y corrección de brazo de palanca (Lever-Arm).

5.3 3. Tareas Detalladas de Implementación

En `include/gpr3d/geo/crs_transform.hpp` y `src/geo/crs_transform.cpp`:

- **Subtarea 3.1: Modelado de estructuras geodésicas:**

```
struct Wgs84Coord { double latitude_deg{0.0}; double longitude_deg{0.0}; double altitude_m
struct EcefCoord { double x_m{0.0}; double y_m{0.0}; double z_m{0.0}; };
struct EnuCoord { double east_m{0.0}; double north_m{0.0}; double up_m{0.0}; };
struct UtmCoord {
    double easting_m{500000.0};
    double northing_m{0.0};
    double elevation_m{0.0};
    int zone{32};
    bool is_northern{true};
};
```

- **Subtarea 3.2: Fórmulas del elipsoide WGS84 a ECEF:** Con $a = 6378137.0$ m, achatamiento $f = 1/298.257223563$, y primera excentricidad al cuadrado $e^2 = 2f - f^2$:

$$N(\varphi) = \frac{a}{\sqrt{1 - e^2 \sin^2 \varphi}} \tag{3}$$

$$X = (N + h)\cos\varphi\cos\lambda, \quad Y = (N + h)\cos\varphi\sin\lambda, \quad Z = (N(1 - e^2) + h)\sin\varphi \tag{4}$$

- **Subtarea 3.3: ECEF a ENU local centrada en $(\varphi_0, \lambda_0, h_0)$:** Multiplicación matricial ortonormal:

$$\begin{bmatrix} E \\ N \\ U \end{bmatrix} = \begin{bmatrix} -\sin\lambda_0 & \cos\lambda_0 & 0 \\ -\sin\varphi_0\cos\lambda_0 & -\sin\varphi_0\sin\lambda_0 & \cos\varphi_0 \\ \cos\varphi_0\cos\lambda_0 & \cos\varphi_0\sin\lambda_0 & \sin\varphi_0 \end{bmatrix} \begin{bmatrix} X - X_0 \\ Y - Y_0 \\ Z - Z_0 \end{bmatrix} \tag{5}$$

- **Subtarea 3.4: Proyección UTM de sexto orden de Krüger:** Implementa `wgs84_to_utm` para determinar automáticamente el huso UTM (1..60), el meridiano central $\lambda_0 = (6 \cdot \text{zone} - 183)^\circ$ y el falso Este (500 000 m).

- **Subtarea 3.5: Corrección de brazo de palanca (*Lever-Arm*):**

$$E_{\text{ant}} = E_{\text{gps}} + \Delta x \sin\psi + \Delta y \cos\psi, \qquad N_{\text{ant}} = N_{\text{gps}} + \Delta x \cos\psi - \Delta y \sin\psi \tag{6}$$

5.4 4. Prueba de Verificación Unitaria

Verifica las coordenadas del punto de referencia geodésico de Berlín: $\varphi = 52.520008^\circ$, $\lambda = 13.404954^\circ$, $h = 34.5$ m.

1. Al convertir a ECEF y retornar a WGS84 (*round-trip*), la discrepancia en latitud y longitud debe ser inferior a 10^{-9} grados (< 0.1 mm de error residual).
2. Al calcular la proyección UTM, el huso debe ser exactamente **33N**, con $E \approx 391777.3$ m y $N \approx 5820074.2$ m.

6 Sprint 4: Receptores GNSS, NMEA-0183 (NMEA GGA) e Interpolación de Trayectoria

6.1 1. Fundamentos y Mapeo Bibliográfico

- ***Mathematik*** **(Arens et al.)**:
  - **Kapitel 24 (*Kurven im Raum*)**: Longitud de arco acumulada $s_i = \sum_{k=1}^{i} \|\mathbf{x}_k - \mathbf{x}_{k-1}\|$, cálculo de rumbo tangencial continuo y curvatura planar.
  - **Kapitel 28 (*Numerik*)**: Interpolación monótona de coordenadas espaciales frente a sellos de tiempo discretos.
- ***Physik*** **(Tipler & Mosca)**:
  - **Kapitel 3 (*Kinematik*)**: Estimación de velocidad instantánea de avance del carrito $v_{\text{cart}} = \Delta s / \Delta t$ y orientación azimutal respecto al Norte verdadero.
- ***C++ professionell anwenden*** **(Breymann)**:
  - **Kapitel 16 (*String- und Stream-Verarbeitung*)**: Tokenización eficiente de cadenas NMEA delimitadas por comas (`std::string_view`, `std::stod`) y manejo de excepciones en conversiones numéricas.

6.2 2. Justificación de Ingeniería: ¿Por qué hay que implementarlo?

1. **Desincronización de Frecuencias de Muestreo GPR vs. GNSS:** Un radar de subsuelo adquiere pulsos a altas frecuencias (por ejemplo, 50 a 100 scans/segundo o mediante una rueda odómetro con 1 scan cada 2 cm), mientras que un receptor GPS/GNSS entrega sentencias NMEA a 1 Hz o como máximo 10 Hz. Esto significa que por cada posición GPS recibida se generan decenas o cientos de trazas de radar. Si no se construye un interpolador espacial robusto por longitud de arco, el 90% de las trazas del radargrama quedarían huérfanas sin georreferenciación.
2. **Peculiaridades de Archivos de Marcas (.dzg):** Equipos como GSSI RADAN almacenan las trazas en un archivo binario `.dzt` y las sentencias GNSS en un archivo de texto adjunto `.dzg`. El archivo `.dzg` intercala etiquetas propietarias como `GSSIS,1420` (indicando que la siguiente sentencia GPS corresponde exactamente a la traza número 1420). Un parser estricto debe extraer estas marcas para sincronizar exactamente la cinemática del levantamiento.

6.3 3. Tareas Detalladas de Implementación

En `include/gpr3d/io/gps_reader.hpp` y `include/gpr3d/geo/trajectory_sync.hpp`:

- **Subtarea 4.1: Parser de tramas NMEA-0183 GPGGA / GNGGA:** Extrae la hora UTC, latitud (formato `ddmm.mmmm` con hemisferio N/S), longitud (formato `dddmm.mmmm` con hemisferio E/W), calidad de fix (1 = GPS, 2 = DGPS, 4 = RTK Fixed, 5 = RTK Float) y elevación sobre el geoide:

```
inline double parse_nmea_coord(const std::string& val, char hem) {
    if (val.empty()) return 0.0;
    double raw = std::stod(val);
    double deg = std::floor(raw / 100.0);
    double min = raw - (deg * 100.0);
    double res = deg + (min / 60.0);
    if (hem == 'S' || hem == 'W') res = -res;
    return res;
}
```

- **Subtarea 4.2: Sincronización de trayectoria `synchronize_trajectory`:** Itera sobre todas las trazas del `BScan`:

  1. Si existen fijaciones GNSS con índice de traza explícito (`trace_index`), realiza una interpolación lineal métrica entre pares consecutivos.
  2. Si no hay índices de traza pero sí marcas de tiempo, interpola por tiempo acumulado.
  3. Si solo hay distancia odrométrica, distribuye los puntos GNSS proporcionalmente a lo largo de la longitud de la trayectoria.
  4. Para cada traza interpolada, calcula el rumbo instantáneo del avance:

$$\psi_i = \text{atan2}(E_{i+1} - E_{i-1},\, N_{i+1} - N_{i-1}) \tag{7}$$

  y aplica la corrección Lever-Arm de la antena.

6.4 4. Prueba de Verificación Unitaria

Configura 3 puntos GNSS: $P_0 = (52.0°, 13.0°)$ en traza 0, $P_1 = (52.001°, 13.0°)$ en traza 500, $P_2 = (52.002°, 13.0°)$ en traza 1000. Verifica que la traza intermedia 250 tiene latitud exactamente igual a $52.0005° \pm 10^{-7\circ}$ y rumbo Norte constante (0°).

7 Sprint 5: Decodificadores I/O de Georradar (GSSI DZT, SEG-Y y MALA RD3)

7.1 1. Fundamentos y Mapeo Bibliográfico

- ***Mathematik* (Arens et al.)**:
  - **Kapitel 1 & 28**: Lectura matricial continua $M \times N$, alineación de bytes y mapeo de memoria sin copias (*zero-copy*).
- ***Physik* (Tipler & Mosca)**:
  - **Kapitel 15 (*Wellen*)**: Tiempo doble de viaje TWT, resolución temporal $\Delta t = T_{\text{range}}/N_{\text{samp}}$ y frecuencia de corte de Nyquist $f_{\text{Nyq}} = 1/(2\Delta t)$.
- ***C++ professionell anwenden* (Breymann)**:

– **Kapitel 16 (*Binäre Dateiverarbeitung*)**: Uso de `std::ifstream(path, std::ios::binary | std::ios::ate)`, validación de tamaño mínimo de archivo y patrones de fábrica polimórficos (`std::unique_ptr<RadarReader>`).

7.2 2. Justificación de Ingeniería: ¿Por qué hay que implementarlo?

1. **Soporte de Formatos Propietarios Industriales:** Cada fabricante de GPR tiene un estándar cerrado. GSSI RADAN utiliza una cabecera binaria de 1024 bytes que puede extenderse a múltiplos de 1024 si se trata de un equipo multicanal (`rh_data`). La profundidad de palabra puede ser de 8 bits (firmas de 0..255), 16 bits sin signo (con sesgo en 32768) o 32 bits enteros signados. SEG-Y utiliza una cabecera textual EBCDIC de 3200 bytes, una cabecera binaria de 400 bytes y cabeceras de traza de 240 bytes con coordenadas escaladas. Si el software no normaliza todos los formatos a un `BScan` con amplitudes en flotante real centradas en cero, ningún filtro matemático posterior funcionará.
2. **Filtrado de Marcadores de Hardware en Muestras 0 y 1:** Los digitalizadores de GSSI (SIR-20, SIR-3000) graban el contador secuencial de scans en la muestra temporal $j = 0$, y en la muestra $j = 1$ graban banderas de hardware (como `0xE4000000` cuando el operador presiona el botón de marca). Si estas dos muestras no se sustituyen por la amplitud temprana continua ($j = 2$), representan un impulso artificial gigantesco que genera artefactos severos de dispersión en la FFT.

7.3 3. Tareas Detalladas de Implementación

En `include/gpr3d/io/dzt_reader.hpp`, `segy_reader.hpp` y `reader.hpp`:

- **Subtarea 5.1: Clase base abstracta y Factory Dispatcher:**

```
class RadarReader {
public:
    virtual ~RadarReader() = default;
    [[nodiscard]] virtual BScan read(const std::filesystem::path& path,
                                     const ReadOptions& options = {}) const = 0;
};

BScan read_radar_file(const std::filesystem::path& path, const ReadOptions& options = {});
```

- **Subtarea 5.2: Lector de GSSI RADAN (`DztReader`):**
  1. Lee los 1024 bytes de cabecera.
  2. Extrae `rh_data` (offset a muestras), `rh_nsamp` (muestras por traza), `rh_bits` (8, 16 o 32), `rhf_range` (ventana temporal ns), y `rhf_spm` (scans por metro).
  3. Detecta el número de canales `rh_nchan` para desempaquetar canales entrelazados.
  4. Itera sobre cada traza y normaliza según `rh_bits`:

$$s_8 = \text{u8} - 128.0, \quad s_{16} = \text{u16_le} - 32768.0, \quad s_{32} = \text{i32_le} \tag{8}$$

  5. Reemplaza muestras 0 y 1 si superan 15× la amplitud temprana del pulso.

- **Subtarea 5.3: Lector de SEG-Y Industrial (`SegyReader`):** Lee la cabecera binaria de 400 bytes (offset 3200), extrae el código de formato de datos (código 1: IBM 360 float, código

5: IEEE 754 float, código 3: entero de 16 bits). Decodifica cada cabecera de traza de 240 bytes aplicando el factor de escala de coordenadas (bytes 68–70) a las posiciones X/Y (bytes 72–88).

7.4 4. Prueba de Verificación Unitaria

Abre el archivo de prueba real del repositorio `parking.nx15.dzt`:

1. Comprueba que no lanza excepciones.
2. Verifica que `bscan.num_traces() > 1000` y `bscan.num_samples() == 1024`.
3. Verifica que $\Delta t = T_{\text{range}}/N_{\text{samp}} > 0.0$ ns (típicamente entre 0.02 ns y 0.05 ns).

8 Sprint 6: Núcleo Espectral FFT y Transformada Analítica de Hilbert

8.1 1. Fundamentos y Mapeo Bibliográfico

- ***Mathematik* (Arens et al.)**:
  - **Kapitel 15 & 16 (*Komplexe Zahlen*)**: Álgebra en el plano complejo $\mathbb{C}$, módulo $|z| = \sqrt{a^2 + b^2}$, argumento atan2(b, a) y raíces de la unidad $e^{-j2\pi/N}$.
  - **Kapitel 18 (*Fourier-Transformationen*)**: Algoritmo rápido de Fourier (FFT), respuesta impulsiva del filtro de Hilbert $h(t) = 1/(\pi t)$, y operador de fase en cuadratura $H(f) = -j\,\text{sgn}(f)$.
- ***Physik* (Tipler & Mosca)**:
  - **Kapitel 15 & 30 (*Wellen und Energie*)**: Densidad de energía instantánea de la onda electromagnética proporcional al cuadrado de la envolvente $A(t)^2$, e independencia respecto a la fase instantánea.
- ***C++ professionell anwenden* (Breymann)**:
  - **Kapitel 14 (*STL-Mathematik*)**: Manejo de números complejos con `std::complex<double>`, y envoltura thread-safe con `std::mutex` para bibliotecas nativas de C (FFTW3).

8.2 2. Justificación de Ingeniería: ¿Por qué hay que implementarlo?

1. **La Ilusión de los Cruces por Cero en la Señal Cruda:** Las ondas de radar son pulsos bipolares o tripolares oscilantes con frecuencias dominantes de cientos de megahertz. Entre una cresta positiva y un valle negativo existe un cruce por cero donde la amplitud momentánea es cero. Si un algoritmo de detección busca reflectores en la señal cruda, interpretará erróneamente que en el cruce por cero no hay ningún objeto enterrado.
2. **Inversión de Polaridad de Fase en Metales frente a Plásticos:** Cuando el radar choca contra un tubo de acero o un cable metálico (conductor eléctrico perfecto, PEC), el coeficiente de reflexión es $R \approx -1$, invirtiendo la fase 180° (el primer pico es negativo). Cuando choca

contra un tubo de polietileno (HDPE) vacío o una cavidad de aire, la onda no se desfasa ($R > 0$, pico positivo). La **envolvente instantánea de Hilbert**:

$$A(t) = |z(t)| = \sqrt{s(t)^2 + \tilde{s}(t)^2} \tag{9}$$

es estrictamente positiva en todos los puntos y representa la energía electromagnética real del reflector sin depender de su polaridad de fase.

8.3 3. Tareas Detalladas de Implementación

En `include/gpr3d/dsp/fft.hpp` y `src/dsp/fft.cpp`:

- **Subtarea 6.1: Algoritmo Radix-2 Cooley-Tukey FFT in-place autónomo:** Permite compilar el software sin depender obligatoriamente de librerías externas. Implementa la permutación de inversión de bits (*bit-reversal permutation*) seguida de los bucles mariposa (*butterfly stages*) con factores de giro (*twiddle factors*) $e^{\pm j2\pi/\text{len}}$.
- **Subtarea 6.2: Wrapper acelerado con FFTW3:** Si se detecta la macro `GPR3D_USE_FFTW3`, ejecuta planes `fftw_plan_dft_1d` protegidos por un `std::mutex` estático para garantizar seguridad multihilo (*thread safety*).
- **Subtarea 6.3: Operador Analítico de Marple (1999) en Frecuencia:** Para una traza real de longitud N:
  1. Calcula la FFT hacia adelante: $S = \text{fft_forward}(s)$.
  2. Multiplica por el operador discreto:

$$H_k = \begin{cases} 1 & \text{si } k = 0 \text{ o } k = N/2 \\ 2 & \text{si } 1 \le k < N/2 \\ 0 & \text{si } N/2 < k < N \end{cases} \tag{10}$$

  3. Calcula la IFFT inversa: $z = \text{fft_inverse}(S')$.
  4. La parte real de $z(t)$ es la señal original $s(t)$, y la parte imaginaria es su transformada de Hilbert $\tilde{s}(t)$.
  5. La envolvente es $A(t) = |z(t)|$ y la fase instantánea es $\phi(t) = \text{atan2}(\text{Im}(z), \text{Re}(z))$.

8.4 4. Prueba de Verificación Unitaria

Evalúa la señal analítica teórica: $s(t) = 3.5\cos(2\pi \cdot 200\ \text{MHz} \cdot t)$. Su envolvente de Hilbert debe ser idénticamente igual a 3.5 constante. Comprueba que para todas las muestras lejos de los bordes ($10\% < j < 90\%$), $|A_j - 3.5| < 10^{-5}$.

9 Sprint 7: Pipeline de Acondicionamiento DSP (Dewow, t0, BP, Fondo, SEC)

9.1 1. Fundamentos y Mapeo Bibliográfico

- ***Mathematik* (Arens et al.)**:

- **Kapitel 18 (*Fourier-Transformationen*)**: Teorema de convolución temporal, filtrado espectral y aliasing por convolución circular en la DFT.

- ***Physik* (Tipler & Mosca)**:
  - **Kapitel 15 & 28 (*Wechselströme und Filter*)**: Deriva inductiva de baja frecuencia (*wow*), alineación del acoplamiento dieléctrico aire-suelo y dispersión de energía geométrica y disipativa.

- ***C++ professionell anwenden* (Breymann)**:
  - **Kapitel 11 (*std::span*)**: Vistas continuas de memoria sin copias ni asignaciones dinámicas repetitivas.

9.2 2. Justificación de Ingeniería: ¿Por qué hay que implementarlo?

El orden de las operaciones en el pipeline de georradar no es arbitrario; está gobernado estrictamente por la física del instrumento y las propiedades de la transformada de Fourier (ver Figura 4):

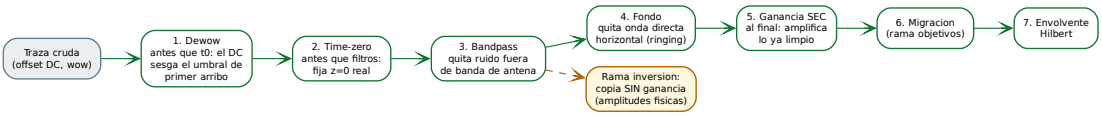


Figure 4: Orden obligatorio del pipeline de procesamiento DSP y bifurcación de la rama de inversión.

1. **¿Por qué Dewow debe ir ANTES de Time-Zero?:** El acoplamiento electromagnético inmediato entre las antenas transmisora y receptora satura el circuito y crea una deriva de baja frecuencia (*inductive wow*) que desplaza la línea base. Si no se sustrae la media móvil en ventana corta, la detección del primer arribo de superficie (cruce por umbral del 15% del pico) se disparará prematuramente en el ruido continuo, corrompiendo la referencia de cota cero del terreno.

2. **¿Por qué Time-Zero debe usar un desvanecimiento coseno (*Cosine-Taper*)?:** Al alinear el terreno en $t = 0$ ns, todas las muestras anteriores al primer arribo se descartan y la traza se rellena con ceros al final. Si la traza termina bruscamente en un valor no nulo, se genera un escalón artificial (*Heaviside step*). Al aplicar la FFT en el siguiente paso, este escalón introduce altas frecuencias falsas (fenómeno de Gibbs) que contaminan el radargrama. Un desvanecimiento coseno de 16 muestras en la cola elimina este salto suavemente.

3. **¿Por qué es indispensable el Zero-Padding al Doble ($2N$) en el Filtro Butterworth?:** El filtrado espectral mediante multiplicación en frecuencia $S(f) \cdot G(f)$ equivale a una **convolución circular** periódica. Si no se rellena con ceros hasta al menos el doble del tamaño de la traza ($2N$), la enorme energía de la onda directa de superficie en $t = 0$ se "envuelve" circularmente y reaparece al final de la traza ($t = t_{\max}$), simulando un reflector profundo inexistente (ver Figura 5).

4. **¿Por qué la Ganancia SEC debe ir al FINAL y mantenerse fuera de la Inversión?:** La compensación esférica y exponencial SEC:

$$g(t) = (1 + \beta t) \cdot \exp(\alpha t) \tag{11}$$

amplifica las señales profundas miles de veces. Si se aplica antes de los filtros, amplificará el ruido de fondo y la deriva DC haciendo imposible su eliminación. Además, la rama de inversión geofísica (Sprint 9) debe trabajar obligatoriamente con trazas **sin ganancia** para poder medir coeficientes de reflexión físicos R reales.

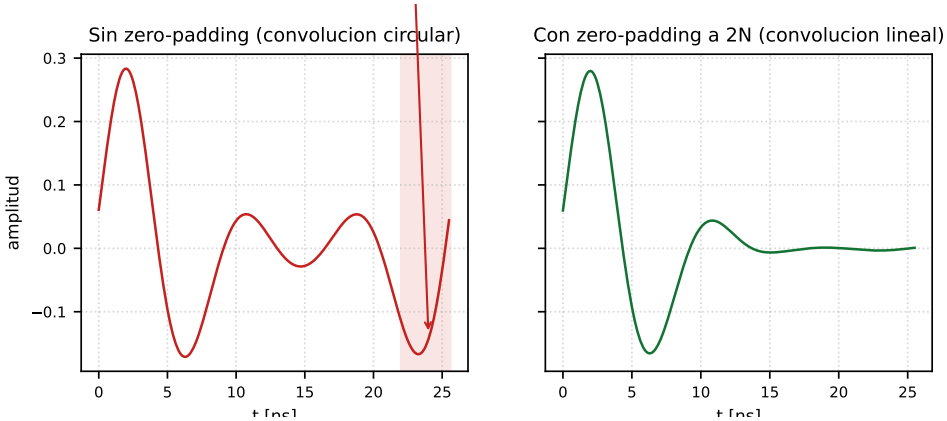


Figure 5: Comparación entre convolución circular (sin padding) y lineal (con padding $2N$).

9.3 3. Tareas Detalladas de Implementación

En `src/dsp/signal_processor.cpp`:

- **Subtarea 7.1: Dewow con Sumas Prefijas en Complejidad $O(N)$:** Calcula el arreglo prefijo $P_{j+1} = P_j + s_j$ y resta la media local en ventana $[-W, +W]$ en tiempo constante por muestra:

```
void SignalProcessor::apply_dewow(BScan& bscan, double window_ns) {
    const double dt = bscan.metadata.sample_interval_ns;
    const size_t half_w = (window_ns > 0.0 && dt > 0.0)
                              ? std::max<size_t>(1, static_cast<size_t>(std::lround(0.5 * w
                              : 0;
    std::vector<double> psum;
    for (auto& tr : bscan.traces) {
        const size_t n = tr.samples.size();
        if (n == 0) continue;
        psum.assign(n + 1, 0.0);
        for (size_t j = 0; j < n; ++j) psum[j + 1] = psum[j] + tr.samples[j];
        for (size_t j = 0; j < n; ++j) {
            const size_t a = (j > half_w) ? (j - half_w) : 0;
            const size_t b = std::min(n - 1, j + half_w);
            const double mean_loc = (psum[b + 1] - psum[a]) / static_cast<double>(b - a + 1
            tr.samples[j] -= mean_loc;
        }
    }
}
```

- **Subtarea 7.2: Time-Zero con Cosine-Taper:** Calcula el promedio apilado de las primeras 40% muestras, detecta el primer cruce que supere el 15% del máximo global (j_0), desplaza a la izquierda y suaviza las últimas 16 muestras con una ventana coseno antes del relleno de ceros.

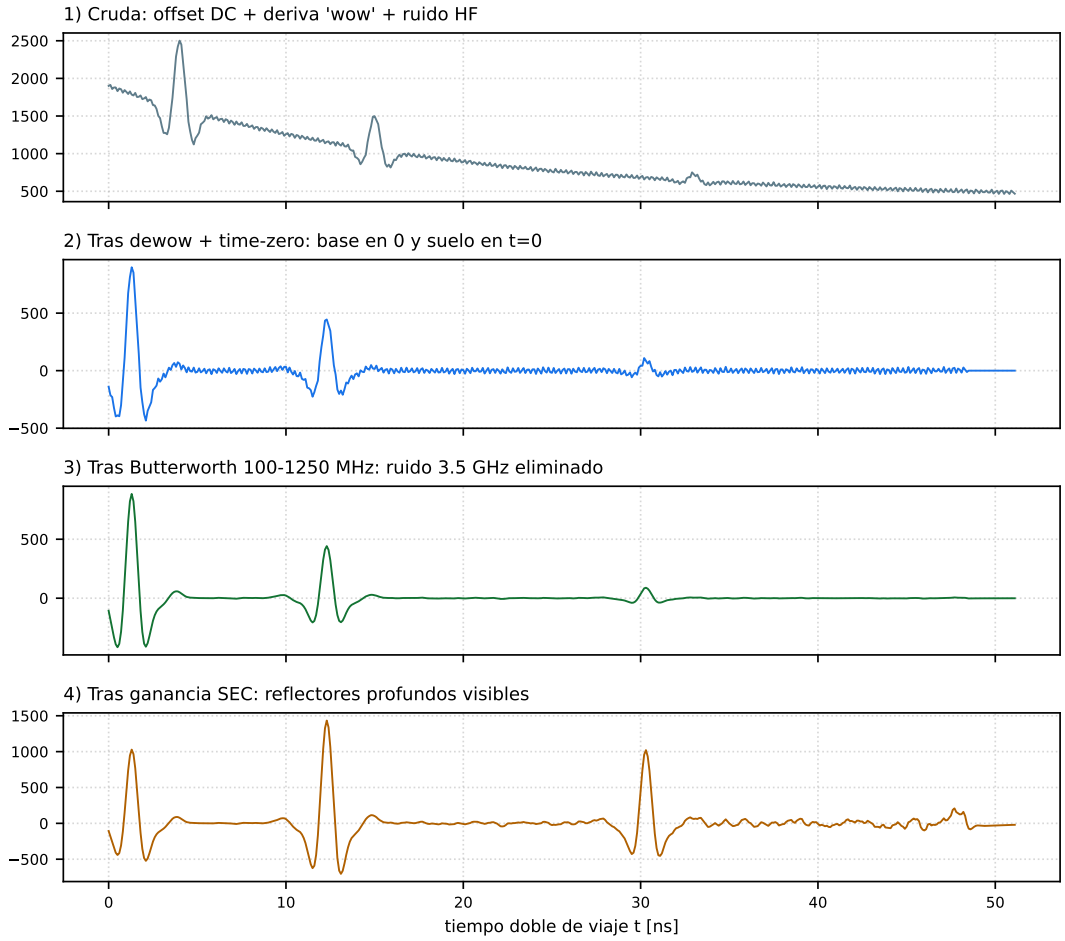


Figure 6: Transformación progresiva de una traza sintética a lo largo de las 4 etapas del pipeline DSP.

- **Subtarea 7.3: Filtro Butterworth Pasabanda con Zero-Padding a $2N$:** Calcula $N_{\text{fft}} = 2^{\lceil \log_2(2N) \rceil}$. Genera la función de ganancia real simétrica de orden $N_B = 4$:

$$G(f) = \frac{1}{\sqrt{1 + (f_{\text{low}}/f)^{2N_B}}} \cdot \frac{1}{\sqrt{1 + (f/f_{\text{high}})^{2N_B}}} \tag{12}$$

  Multiplica el espectro FFT complejo y aplica la IFFT inversa, conservando solo las primeras N muestras reales.

- **Subtarea 7.4: Sustracción de Fondo (Background Removal):** Calcula la traza media espacial $\bar{s}_j = \frac{1}{M} \sum_{i=1}^{M} s_{i,j}$ y réstala de todas las trazas para suprimir el acoplamiento directo horizontal transmisor-receptor.

- **Subtarea 7.5: Ganancia SEC con Techo de Saturación (*Gain Ceiling*):** Multiplica por $(1 + \beta t)\exp(\alpha t)$ con un límite superior de 60 dB para evitar amplificar el ruido térmico profundo.

9.4 4. Prueba de Verificación Unitaria

Aplica el pipeline DSP completo sobre una traza de prueba con deriva DC de 500.0, señal útil en 400 MHz y ruido de alta frecuencia en 3000 MHz. Verifica que la deriva DC se reduce en más del 99.9%, la señal de 400 MHz mantiene su amplitud original ±5%, y el ruido de 3000 MHz se atenúa en más de 30 dB.

10 Sprint 8: Migración 2D de Kirchhoff y Nubes de Puntos 3D Georreferenciadas

10.1 1. Fundamentos y Mapeo Bibliográfico

- ***Mathematik*** **(Arens et al.)**:
  - **Kapitel 24 (*Flächenintegrale*)**: Integrales de superficie y formulación de la integral de frontera de Helmholtz-Kirchhoff.
  - **Kapitel 28 (*Interpolation*)**: Interpolación lineal sub-muestra entre muestras temporales discretas continuas.
- ***Physik*** **(Tipler & Mosca)**:
  - **Kapitel 15 (*Wellenbeugung und Huygensscher Prinzip*)**: Principio de Huygens: cada punto difractor en el subsuelo reemite como una fuente secundaria esférica. La migración representa el colapso espacial retropropagado de los frentes de onda difractados.
- ***C++ professionell anwenden*** **(Breymann)**:
  - **Kapitel 7 & 8 (*Klassen und Move-Semantik*)**: Diseño eficiente de la estructura de nube de puntos `PointCloud3D` con reserva contigua para evitar millones de micro-asignaciones.

10.2 2. Justificación de Ingeniería: ¿Por qué hay que implementarlo?

1. **La Distorsión Hiperbólica de Tubos y Cavidades:** Debido a la apertura angular del cono de radiación de la antena electromagnética, un tubo enterrado de apenas 10 cm de diámetro no aparece como un punto en el radargrama, sino como una hipérbola gigante que abarca varios metros de ancho. Si intentamos generar una nube de puntos o una malla 3D directamente desde el radargrama crudo, obtendremos una superficie curva gigantesca en lugar de una tubería cilíndrica.
2. **Colapso Coherente de Energía con Kirchhoff:** La migración de Kirchhoff suma coherentemente las amplitudes a lo largo de la hipérbola teórica de difracción:

$$t(x) = \frac{2}{v}\sqrt{(x - x_0)^2 + z_0^2} \tag{13}$$

colapsando toda la energía dispersa de la hipérbola al punto focal real (x_0, z_0) y suprimiendo las colas laterales mediante el factor de oblicuidad $\cos\theta = z_0/R$.

10.3 3. Tareas Detalladas de Implementación

En `src/dsp/signal_processor.cpp` y `src/dsp/interpreter.cpp`:

- **Subtarea 8.1: Algoritmo de Migración 2D de Kirchhoff en Tiempo:** Para cada traza i_0 en x_0 y cada profundidad de muestra j_0 (tiempo $t_0 = j_0\Delta t$, profundidad $z_0 = \frac{1}{2}vt_0$):
  1. Determina el rango de trazas dentro de la apertura horizontal $[x_0 - L_{\text{ap}},\, x_0 + L_{\text{ap}}]$.
  2. Para cada traza vecina i en posición x_i, calcula la distancia oblicua $R = \sqrt{(x_i - x_0)^2 + z_0^2}$.
  3. Calcula el tiempo de viaje teórico $t_{\text{diff}} = 2R/v$ y su índice continuo $s_{\text{idx}} = t_{\text{diff}}/\Delta t$.
  4. Realiza interpolación lineal entre las muestras enteras $k = \lfloor s_{\text{idx}} \rfloor$ y $k + 1$.
  5. Pondera por el factor de oblicuidad $\cos\theta = z_0/R$ y suma a la muestra de salida.
- **Subtarea 8.2: Conversión TWT a Profundidad y Nube de Puntos Georreferenciada:** En `RadarInterpreter::transform_to_3d`, convierte cada muestra o pico de envolvente en un `Point3D` con coordenadas geográficas WGS84, proyección métrica UTM y coordenadas locales ENU, asignando la profundidad $z = \frac{1}{2}v \cdot \text{twt}$ como cota negativa bajo el terreno.

10.4 4. Prueba de Verificación Unitaria

Aplica la migración de Kirchhoff sobre el radargrama sintético del Sprint 2 (hipérbola centrada en $x = 2.5$ m, $z = 0.8$ m). Verifica que tras la migración, la energía de las ramas de la hipérbola se reduce en más del 80% y el pico máximo de la envolvente migrada se concentra a menos de 5 cm del centro verdadero ($x = 2.5, z = 0.8$).

11 Sprint 9: Inversión Geofísica Subterránea (Semblanza, Fresnel y ASTM D6432)

11.1 1. Fundamentos y Mapeo Bibliográfico

- ***Mathematik* (Arens et al.)**:

  - **Kapitel 28 (*Ausgleichsrechnung*)**: Métrica de semblanza de Neidell-Taner (energía apilada sobre energía individual normalizada), ajuste parabólico de 3 puntos alrededor de un máximo para resolución submalla, y partición en tiempo lineal $O(N)$ con `std::nth_element`.

- ***Physik* (Tipler & Mosca)**:
  - **Kapitel 30, 31 & 32 (*Elektromagnetische Wellen und Reflexion*)**: Coeficientes de reflexión de Fresnel en incidencia normal, impedancia de onda $\eta = \sqrt{\mu/\varepsilon}$, velocidad electromagnética $v = c_0/\sqrt{\varepsilon_r}$, tangente de pérdidas $\tan\delta$, y corrimiento del centroide espectral (Quan-Harris).

- ***C++ professionell anwenden* (Breymann)**:
  - **Kapitel 13 & 14 (*STL-Algorithmen*)**: Algoritmos de selección rápida y ordenación (`std::nth_element`, `std::sort`, `std::clamp`).

11.2 2. Justificación de Ingeniería: ¿Por qué hay que implementarlo?

1. **El Peligro de Calibrar la Velocidad Manualmente:** Los operadores suelen asumir un valor fijo genérico de permitividad (p. ej. $\varepsilon_r = 9.0 \implies v = 0.10$ m/ns). Sin embargo, si el asfalto está seco ($\varepsilon_r = 4.0 \implies v = 0.15$ m/ns) o el terreno está empapado ($\varepsilon_r = 25.0 \implies v = 0.06$ m/ns), el error de profundidad supera el 100%, provocando que excavadoras rompan tuberías que estaban más cerca o más profundas de lo estimado. El **análisis de semblanza en hipérbolas** deduce la velocidad in situ a partir de la curvatura de la onda sin perforar el pavimento (Figura 7).

2. **Estratigrafía Multicapa de Fresnel en Carreteras:** El suelo bajo un pavimento no es uniforme; está formado por capas sucesivas (asfalto de rodadura, base granular de grava y subrasante arcillosa). Con la ley de Fresnel en incidencia normal:

$$R_k = \frac{\sqrt{\varepsilon_{r,k}} - \sqrt{\varepsilon_{r,k+1}}}{\sqrt{\varepsilon_{r,k}} + \sqrt{\varepsilon_{r,k+1}}} \implies \varepsilon_{r,k+1} = \varepsilon_{r,k}\left(\frac{1 - R_k}{1 + R_k}\right)^2 \tag{14}$$

   es posible deducir la permitividad y el tipo de material de cada estrato hacia abajo sucesivamente (Figura 8).

3. **Clasificación Automática de Materiales según ASTM D6432:** Los clientes y los sistemas GIS no necesitan solo valores numéricos abstractos de ε_r; necesitan saber si el estrato es *Asphalt_Dry*, *Gravel_Dry*, *Concrete* o *Clay_Wet*, y si el conducto es metálico o polimérico (PVC/HDPE).

11.3 3. Tareas Detalladas de Implementación

En `include/gpr3d/dsp/soil_inversion.hpp`, `src/dsp/soil_inversion.cpp` y `src/geo/material_db.cpp`:

- **Subtarea 9.1: Ajuste de Velocidad por Semblanza de Neidell-Taner:** Para un ápice (x_0, t_0), evalúa una grilla de velocidades $v_g \in [0.05, 0.20]$ m/ns con paso $\Delta v = 0.002$ m/ns.

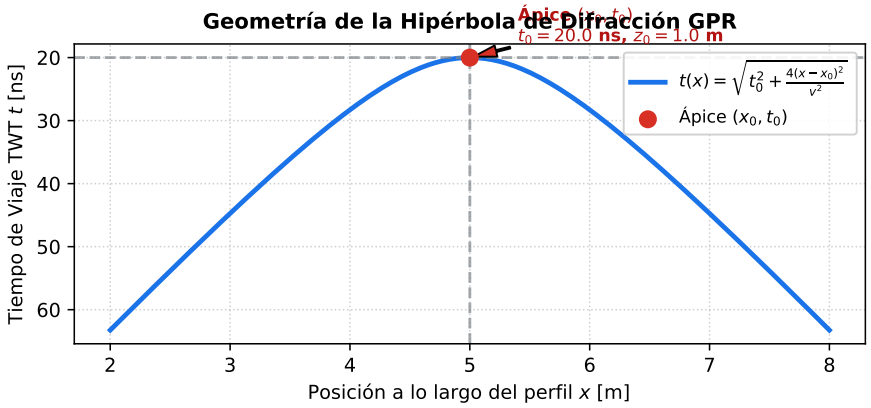


Figure 7: Geometría hiperbólica del tiempo de viaje TWT y escaneo de velocidad v.

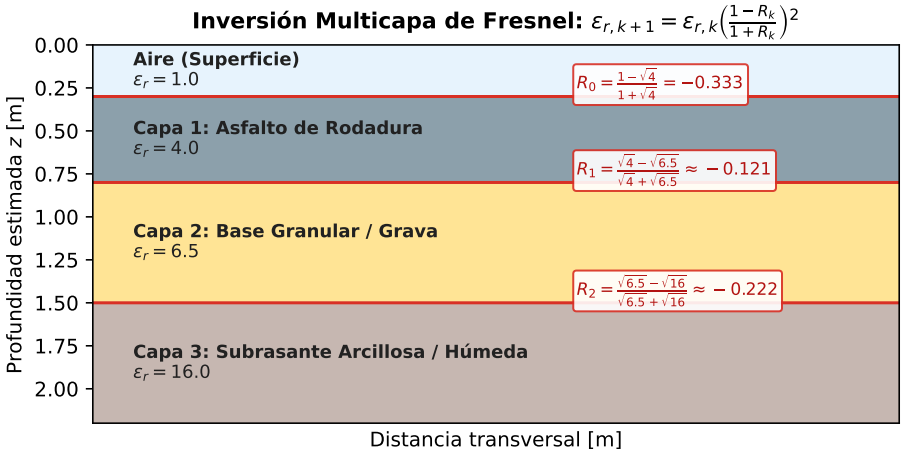


Figure 8: Inversión estratigráfica multicapa de Fresnel.

Calcula la semblanza $S(v)$ sumando amplitudes sobre la hipérbola en una semiventana temporal $w \in [-M, +M]$:

$$S(v) = \frac{\sum_{w=-M}^{M} \left[\sum_{i=1}^{K} s_i(t_i(v) + w\Delta t)\right]^2}{K \sum_{w=-M}^{M} \sum_{i=1}^{K} \left[s_i(t_i(v) + w\Delta t)\right]^2} \tag{15}$$

Encuentra el índice óptimo g^* y aplica refinamiento parabólico submalla de tres puntos:

$$\delta = \frac{1}{2} \frac{S_{g^*-1} - S_{g^*+1}}{S_{g^*-1} - 2S_{g^*} + S_{g^*+1}}, \qquad v^* = v_{g^*} + \delta \Delta v, \qquad \varepsilon_r = \left(\frac{c_0}{v^*}\right)^2 \tag{16}$$

- **Subtarea 9.2: Estimador de Atenuación Quan-Harris:** Calcula el corrimiento hacia frecuencias bajas del centroide espectral entre ventanas someras y profundas para estimar la atenuación intrínseca α (dB/m), la pérdida tangente $\tan\delta$ y la conductividad aparente σ (mS/m).
- **Subtarea 9.3: Base de Datos Litológica ASTM D6432 (`MaterialDatabase`):** Implementa el clasificador bayesiano `classify_soil_layer` que asigna probabilidades evaluando la distancia logarítmica respecto a los rangos permitidos de permitividad y conductividad.

11.4 4. Prueba de Verificación Unitaria

1. Prueba analítica de Fresnel: Con $\varepsilon_{r,1} = 4.0$ y $R = -0.333333$, la función `invert_fresnel_permittivity` debe devolver exactamente $\varepsilon_{r,2} = 16.000 \pm 0.001$.
2. Clasificador ASTM: Para $\varepsilon_r = 4.2$, $\sigma = 0.5$ mS/m en superficie ($z = 0$ m), el material debe ser clasificado como `Asphalt_Dry` con probabilidad $> 35\%$.

12 Sprint 10: Reconstrucción Semántica 3D y Generación de Mallas

12.1 1. Fundamentos y Mapeo Bibliográfico

- ***Mathematik* (Arens et al.)**:
  - **Kapitel 24 & 25 (*Kurven und Flächen*)**: Triedro móvil de Frenet-Serret ($\mathbf{T}, \mathbf{N}, \mathbf{B}$), vector tangente $\mathbf{T} = \mathbf{c}'(s)$, vector normal $\mathbf{N}$ y vector binormal $\mathbf{B} = \mathbf{T} \times \mathbf{N}$. Parametrización cilíndrica espacial:
$$\mathbf{r}(s, \theta) = \mathbf{c}(s) + r\left(\cos\theta \mathbf{N}(s) + \sin\theta \mathbf{B}(s)\right) \tag{17}$$
- ***Physik* (Tipler & Mosca)**:
  - **Kapitel 15**: Continuidad geométrica y consistencia espacial de reflectores enterrados.
- ***C++ professionell anwenden* (Breymann)**:
  - **Kapitel 7 & 8**: Mallas triangulares indexadas con normales unitarias suaves y orientación antihoraria (CCW).

12.2 2. Justificación de Ingeniería: ¿Por qué hay que implementarlo?

1. **De Nubes Dispersas a Objetos CAD/BIM Continuos:** Los ingenieros civiles no pueden diseñar obras basándose en millones de puntos aislados de una nube de puntos. Necesitan objetos CAD/BIM semánticos sólidos: un conducto cilíndrico cerrado con radio y eje central, o superficies de estrato de pavimento trianguladas.

2. **Por qué es Indispensable el Triedro de Frenet-Serret en Tubos 3D:** Al extruir un tubo a lo largo de una línea 3D con curvas, si se proyectan los vértices del cilindro usando un plano cartesiano estático fijo (p. ej. XY), cuando la tubería gira o cambia de pendiente vertical, el cilindro colapsa sobre sí mismo (*gimbal lock*) y sufre torsiones bruscas. El marco local móvil $(\mathbf{T}, \mathbf{N}, \mathbf{B})$ se desplaza y gira suavemente acompañando a la curva, garantizando una sección transversal perfectamente circular en todo momento (Figura 9).

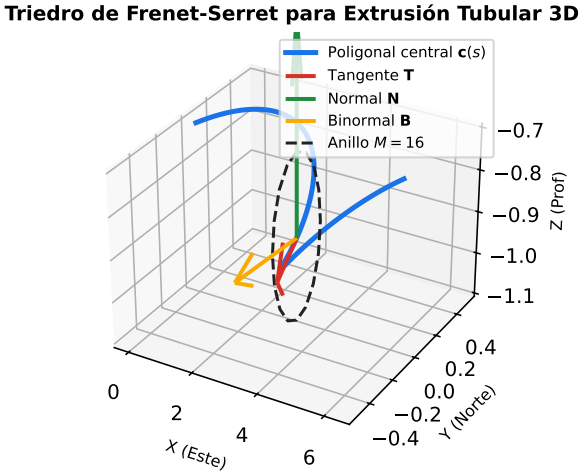


Figure 9: Triedro móvil de Frenet-Serret para la extrusión tubular cilíndrica de conductos 3D.

12.3 3. Tareas Detalladas de Implementación

En `include/gpr3d/scene/objects_3d.hpp` y `src/dsp/object_transformer.cpp`:

- **Subtarea 10.1: Definición de estructuras de escena 3D:** Define `Vec3`, `ColorRgba`, `Mesh3D`, `ConduitObject3D`, `LayerHorizonObject3D`, `AnomalyObject3D` y `Scene3D`.

- **Subtarea 10.2: Extrusión tubular `build_tube_mesh`:** Dada una secuencia de puntos del eje central $\mathbf{c}_i$, radio r, y M sectores radiales:

  1. Para cada punto, calcula el vector tangente $\mathbf{T}_i = \text{norm}(\mathbf{c}_{i+1} - \mathbf{c}_{i-1})$.
  2. Elige un vector vertical auxiliar $\mathbf{u} = (0, 0, 1)$ (o $(0, 1, 0)$ si $\mathbf{T}$ es casi vertical).
  3. Calcula la normal $\mathbf{N} = \text{norm}(\mathbf{T} \times \mathbf{u})$ y la binormal $\mathbf{B} = \mathbf{T} \times \mathbf{N}$.
  4. Genera los M vértices del anillo circular:

$$\mathbf{v}_{i,j} = \mathbf{c}_i + r\left(\cos\theta_j \mathbf{N} + \sin\theta_j \mathbf{B}\right), \quad \theta_j = \frac{2\pi j}{M} \tag{18}$$

  5. Ensambla los triángulos entre anillos adyacentes con orientación antihoraria (CCW): $(v_{i,j},\, v_{i+1,j},\, v_{i+1,j+1})$ y $(v_{i,j},\, v_{i+1,j+1},\, v_{i,j+1})$.

- **Subtarea 10.3: Triangulación de Horizontes y Esferas de Anomalías:** Implementa `build_surface_from_profiles` para tejer superficies de contacto entre perfiles paralelos, y `build_ellipsoid_mesh` para cavidades y bloques de roca.

12.4 4. Prueba de Verificación Unitaria

Para un eje recto de 5 puntos con radio 0.1 m y 16 sectores: Verifica que la malla tiene exactamente $5 \times 16 = 80$ vértices y $4 \times 32 = 128$ triángulos, y que ninguna cara triangular tiene área nula.

13 Sprint 11: Exportadores e Interoperabilidad (glTF 2.0 Binary, GeoJSON 3D, HTML)

13.1 1. Fundamentos y Mapeo Bibliográfico

- ***Mathematik* (Arens et al.)**:
  - **Kapitel 7 (*Lineare Abbildungen*)**: Transformaciones de coordenadas afines 4×4 y renderizado proyectivo.
- ***Physik* (Tipler & Mosca)**:
  - **Kapitel 31 (*Optik*)**: Modelos de materiales con renderizado basado en la física (PBR: rugosidad *roughness*, metalicidad *metallic* y albedo difuso).
- ***C++ professionell anwenden* (Breymann)**:
  - **Kapitel 16 (*Binäre Dateiverarbeitung*)**: Serialización estructurada de especificaciones abiertas (Khronos glTF 2.0 y GeoJSON RFC 7946).

13.2 2. Justificación de Ingeniería: ¿Por qué hay que implementarlo?

1. **El Estándar Khronos glTF 2.0 Binary (.glb) como Formato Universal:** Los formatos antiguos como OBJ o DXF no soportan materiales PBR modernos, o son formatos de texto gigantescos que tardan minutos en abrirse. El formato binario `.glb` empaqueta en un único archivo la descripción de nodos y materiales en JSON (*Chunk 0*) junto con los búferes binarios directos de vértices y normales (*Chunk 1*) alineados a 4 bytes, permitiendo que la GPU cargue la geometría en milisegundos sin necesidad de parsear texto.
2. **Georreferenciación en Sistemas GIS con GeoJSON 3D:** Los organismos públicos y empresas de servicios públicos (agua, gas, electricidad, fibra óptica) gestionan sus redes en QGIS o ArcGIS. Un archivo GeoJSON 3D con cotas negativas en formato RFC 7946 permite superponer las tuberías detectadas directamente sobre el mapa municipal.
3. **Cuadros de Mando Web Interactivos sin Instalación:** Entregar un archivo HTML interactivo con Three.js permite que clientes, directores de obra o topógrafos inspeccionen la escena 3D orbital y el radargrama B-scan sincronizado desde cualquier navegador web o smartphone sin instalar software adicional.

13.3 3. Tareas Detalladas de Implementación

En `src/io/scene_exporter.cpp`:

- **Subtarea 11.1: Serializador Khronos glTF 2.0 Binary (.glb):**
  1. Escribe la cabecera de 12 bytes: `magic = 0x46546C67` ("glTF"), `version = 2`, y `total_length`.
  2. Rellena el Chunk 0 (JSON de escena, materiales geológicos PBR, nodos) con espacios ASCII (`0x20`) hasta un múltiplo exacto de 4 bytes. Escribe el encabezado del Chunk (`length`, tipo `0x4E4F534A` "JSON").
  3. Rellena el Chunk 1 (Búfer binario con vértices en float IEEE 754 y triángulos en uint32_t) con ceros (`0x00`) hasta múltiplo de 4 bytes. Escribe el encabezado (`length`, tipo `0x004E4942` "BIN").
- **Subtarea 11.2: Serializador GeoJSON 3D (RFC 7946):** Escribe una `FeatureCollection` donde cada conducto es un `LineString` con coordenadas `[lon, lat, alt_m]` y atributos de material clasificado, diámetro estimado y nivel de confianza.
- **Subtarea 11.3: Cuadro de mando interactivo HTML5 autónomo:** Genera un archivo HTML autocontenido con Three.js embebido, controles orbitales (`OrbitControls`), radargrama sincronizado y panel lateral con la columna litológica.

13.4 4. Prueba de Verificación Unitaria

Valida el archivo `.glb` generado usando un validador oficial Khronos o Blender: La escena debe abrirse sin advertencias de alineación de búfer, con las normales orientadas hacia el exterior y los materiales geológicos diferenciados.

14 Sprint 12: Integración CLI, Analítica Gráfica y Validación Real

14.1 1. Fundamentos y Mapeo Bibliográfico

- ***Mathematik* (Arens et al.)**:
  - **Kapitel 28**: Análisis estadístico de fluctuaciones de señal (media, desviación estándar, asimetría y curtosis).
- ***Physik* (Tipler & Mosca)**:
  - **Kapitel 15**: Interpretación integral de radargramas de campo reales.
- ***C++ professionell anwenden* (Breymann)**:
  - **Kapitel 1 (*Kommandozeilen-Schnittstelle*)**: Parsing de argumentos de línea de comandos (`argc`, `argv`) y manejo robusto de excepciones en el punto de entrada (`main`).

14.2 2. Justificación de Ingeniería: ¿Por qué hay que implementarlo?

Una biblioteca geofísica sin herramientas de línea de comandos ni generadores de informes gráficos resulta inútil para la operación en campo. El CLI `gpr3d` consolida todos los sprints previos en una suite unificada con subcomandos claros:

- `gpr3d inspect`: Diagnóstico geofísico rápido de cabeceras, velocidades e inversión litológica sin procesar todo el levantamiento.
- `gpr3d process`: Acondicionamiento de señal por lotes.
- `gpr3d transform`: Reconstrucción semántica 3D y exportación a GLB, GeoJSON y HTML.
- `gpr3d plot`: Renderizado de radargramas de alta resolución y analíticas de fluctuación con Cairo.

14.3 3. Tareas Detalladas de Implementación

En `apps/gpr3d_cli.cpp` y `src/viz/fluctuation_plotter.cpp`:

- **Subtarea 12.1: CLI Principal y Subcomandos:** Implementa el despacho de comandos, validando argumentos y capturando excepciones con mensajes descriptivos.
- **Subtarea 12.2: Generador de Gráficos de Fluctuación con Cairo:** Crea radargramas en escala de grises o paleta geofísica, perfiles de energía instantánea de Hilbert y gráficos de atenuación en PNG y SVG vectorial.
- **Subtarea 12.3: Validación contra Archivos Reales del Repositorio:** Ejecuta el pipeline completo contra `parking.nx15.dzt` y `ds010_A.DZT`.

14.4 4. Prueba de Verificación Unitaria

Ejecuta en consola: `./build/gpr3d inspect parking.nx15.dzt` Verifica que el informe en terminal reporta:

- Frecuencia nominal de antena coherente con el modelo (p. ej. 1600 MHz para NX15).
- Detección de ápices hiperbólicos con velocidades calculadas entre 0.08 m/ns y 0.15 m/ns.
- Clasificación litológica dominante acorde a pavimento urbano (`Asphalt_Dry` y `Gravel_Dry`).

15 Conclusiones y Hoja de Ruta de Especialización Profesional

Siguiendo paso a paso los 13 sprints desarrollados en esta guía técnica, el desarrollador habrá replicado de forma autónoma una suite completa de software geofísico en C++20 moderno, adquiriendo dominio en:

1. **Arquitectura de C++20 de Alto Rendimiento:** Abstracciones de coste cero, gestión de memoria RAII sin fugas, seguridad de tipos y serialización binaria de bajo nivel.
2. **Procesamiento Digital de Señales (DSP):** Transformada de Fourier FFT, señal analítica de Hilbert, filtrado Butterworth de fase cero y compensación de atenuación.
3. **Geofísica e Inversión Subterránea:** Análisis de semblanza en hipérbolas, inversión de reflectancias de Fresnel y clasificación litológica ASTM D6432.

4. **Geodesia Espacial y Computación Gráfica 3D:** Transformaciones geodésicas (WGS84, ECEF, ENU, UTM), extrusión tubular con el triedro móvil de Frenet-Serret y empaquetado de escenas Khronos glTF 2.0 con materiales PBR.

Proyecto `SignalHandler / gpr3d` • Noviembre 2026

Documento técnico y académico de ingeniería inversa y desarrollo en C++20.